



GUÍA PRÁCTICA 05 APLICACIÓN DE AUTÓMATAS – AFD & AFN

Objetivo General:

Implementar autómatas finitos en C++ para el reconocimiento de patrones utilizados en compiladores.

Objetivos Específicos:

- Comprender el funcionamiento de los autómatas finitos.
- Diseñar tablas de transición.
- Implementar AFD en C++.
- Reconocer cadenas válidas.
- Aplicar autómatas al análisis léxico.

FUNDAMENTO TEÓRICO

Definición de Autómata Finito

Un autómata finito se define mediante la quintupla:

$$A = (Q, \Sigma, \delta, q_0, F)$$

Donde:

Símbolo Descripción

Q	Conjunto de estados
Σ	Alfabeto
δ	Función de transición
q_0	Estado inicial
F	Estados finales

TIPOS DE AUTÓMATAS

1. Autómata Finito Determinista (AFD)

Características:

- Solo existe una transición por símbolo.
- No existen transiciones vacías.
- Cada entrada conduce a un único estado.

2. Autómata Finito No Determinista (AFN)

Características:

- Puede existir más de una transición.
- Puede haber transiciones ϵ .

AUTÓMATA QUE RECONOCE CADENAS TERMINADAS EN 01

Lenguaje

$$L = \{ w \mid w \text{ termina en } 01 \}$$

DIAGRAMA DE ESTADOS

$q_0 \xrightarrow{0} q_1$

$q_1 \xrightarrow{1} q_0$

q1--0--> q1

q1--1--> q2

q2--0--> q1

q2--1--> q0

Estado final: q2

TABLA DE TRANSICIÓN

Estado 0 1

q0 q1 q0

q1 q1 q2

q2 q1 q0

1. IMPLEMENTACIÓN EN C++

```
#include <iostream>
```

```
#include <string>
```

```
using namespace std;
```

```
int main() {  
    string cadena;  
    int estado = 0;  
    cout << "Ingrese cadena binaria: ";  
    cin >> cadena;  
    for(char c : cadena) {  
        switch(estado) {  
            case 0:  
                if(c == '0')  
                    estado = 1;  
                else if(c == '1')  
                    estado = 0;  
                else {  
                    cout << "Simbolo invalido" << endl;  
                    return 0;  
                }  
                break;  
            case 1:  
                if(c == '0')  
                    estado = 1;  
                else if(c == '1')  
                    estado = 2;  
                else {  
                    cout << "Simbolo invalido" << endl;  
                    return 0;  
                }  
                break;  
        }  
    }  
}
```



case 2:

```
if(c == '0')
    estado = 1;
else if(c == '1')
    estado = 0;
else {
    cout << "Simbolo invalido" << endl;
    return 0;
}
break;
}
```

```
if(estado == 2)
    cout << "Cadena aceptada" << endl;
else
    cout << "Cadena rechazada" << endl;
```

```
return 0;
}
```

Variables utilizadas

Variable Función

cadena Almacena la entrada

estado Controla el estado actual

c Recorre cada símbolo

Funcionamiento

El programa:

1. Lee una cadena.
2. Recorre símbolo por símbolo.
3. Cambia de estado según la tabla de transición.
4. Verifica si termina en un estado final.

2. AUTÓMATA QUE RECONOCE NÚMERO PAR DE CEROS

Lenguaje

$L = \{ w \mid w \text{ tiene número par de ceros} \}$

TABLA DE TRANSICIÓN

Estado 0 1

q0 q1 q0

q1 q0 q1

Estado final: q0

IMPLEMENTACIÓN EN C++

```
#include <iostream>
#include <string>
using namespace std;
```

```
int main() {
    string cadena;
    int estado = 0;
    cout << "Ingrese cadena binaria: ";
    cin >> cadena;
    for(char c : cadena) {
        if(c == '0') {
            if(estado == 0)
                estado = 1;
            else
                estado = 0;
        }
        else if(c == '1') {
            continue;
        }
        else {
            cout << "Error" << endl;
            return 0;
        }
    }
    if(estado == 0)
        cout << "Cadena aceptada" << endl;
    else
        cout << "Cadena rechazada" << endl;
    return 0;
}
```

3. RECONOCIMIENTO DE IDENTIFICADORES

Regla léxica

Un identificador:

- Debe iniciar con letra.
- Puede contener letras y números.

Ejemplos válidos:

- variable
- dato1
- sumaTotal

Ejemplos inválidos:

- 1dato
- @valor

IMPLEMENTACIÓN EN C++

```
#include <iostream>
```

```
#include <cctype>
#include <string>
using namespace std;

int main() {

    string palabra;
    cout << "Ingrese identificador: ";
    cin >> palabra;

    if(!isalpha(palabra[0])) {
        cout << "Identificador invalido" << endl;
        return 0;
    }

    for(int i = 1; i < palabra.length(); i++) {

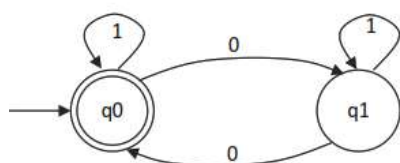
        if(!isalnum(palabra[i])) {
            cout << "Identificador invalido" << endl;
            return 0;
        }
    }
    cout << "Identificador valido" << endl;
    return 0;
}
```

APLICACIÓN EN COMPILADORES

Los compiladores utilizan autómatas para:

Elemento	Ejemplo
Identificadores	variable
Números	123
Operadores	+ - * /
Palabras reservadas	if, while
Delimitadores	;, () {}
identificador = false;	

Ejemplo de programación de un AFD



```
#include<iostream>
using namespace std;
```



```
int main(){
string cadena;
while(1){
    cout<<"\n\nIngrese una cadena de 0s y 1s : ";
    getline(cin,cadena);
    int inicio=1, aceptado=1;
    int estado=inicio;
    bool fin=false;
    int contador=0;

    while(fin==false){
        if(contador>cadena.length()-1){
            fin=true;
            break;
        }
        if(estado==1){
            if(cadena[contador]=='1') estado=1;
            if(cadena[contador]=='0') estado=2;
            contador++;
            continue;
        }
        if(estado==2){
            if(cadena[contador]=='1') estado=2;
            if(cadena[contador]=='0') estado=1;
            contador++;
            continue;
        }
    }
    if(estado==aceptado) {cout<<"La cadena "<<cadena<<" es aceptada "<<endl;}
    else{cout<<"La cadena "<<cadena<<" es rechazada"<<endl;}
}
return 0;
}
```

DESARROLLO DE LA PRÁCTICA

Actividad 1

Construir un autómata que acepte cadenas binarias terminadas en 10.

Actividad 2

Implementar un programa que valide correos electrónicos simples.

Actividad 3

Diseñar un autómata para reconocer números reales.

Actividad 4

Modificar el mini analizador léxico para reconocer:

- while
- if
- else
- for

PREGUNTAS DE REFLEXIÓN

1. ¿Cuál es la diferencia entre AFD y AFN?
2. ¿Por qué los autómatas son importantes en compiladores?
3. ¿Qué función cumple el análisis léxico?
4. ¿Cómo se implementa una tabla de transición en C++?
5. ¿Qué ventajas tiene utilizar autómatas?

EJERCICIOS PROPUESTOS

Ejercicio 1

Construir un AFD para el lenguaje:

$L = \{ w \mid w \text{ contiene la subcadena } 101 \}$

Ejercicio 2

Implementar en C++ un autómata que reconozca:

- números enteros
- números reales
- operadores aritméticos

Ejercicio 3

Diseñar un autómata que valide contraseñas.

Condiciones:

- mínimo 8 caracteres
- una mayúscula
- un número

Ejercicio 4

Construir la tabla de transición para:

$L = \{ w \mid w \text{ tiene cantidad impar de unos} \}$

BIBLIOGRAFÍA

1. Aho, Alfred V. — Compilers: Principles, Techniques and Tools.
2. Hopcroft, John — Introduction to Automata Theory.
3. Loudon, Kenneth — Compiler Construction.
4. Deitel — C++ How to Program.
5. Levine — Flex & Bison.